Assignment 2: AI-Powered Shopping Assistant

Problem Statement: Build a POC for Conversational AI Shopping Assistant for an e-commerce platform using Retrieval-Augmented Generation (RAG). The application should help customers search products, check order status, and place purchases through a chat-based interface. The assistant must use explicit backend function or tool calling for all actions, so responses are reliable, auditable, and do not rely on LLM-generated guesses.

Submission format	Source code repository + written materials + reproducible instructions
Primary dataset	Pick any of the public datasets from Kaggle or Github or generate synthesized data.
LLM	Use any LLM

General Submission Rules

- Use only public, synthetic, or clearly de-identified datasets. Do not use protected health information (PHI) or any private/company-confidential data.
- Your solution must run from a clean environment using the commands you document in README.
- You may use Python and any common open-source library. Do not require paid tools or paid cloud services.
- Include a short design rationale, assumptions, and “what I would improve with more time” note in your submission.
- Include meaningful commit history that reflects how the solution evolved; avoid submitting a single final dump of code.

Assignment Objective

- This assignment evaluates GenAI engineering capability, including prompt design, tool/function orchestration, backend integration (ex: FastAPI), and user-centric AI workflow implementation.
- Because the role operates in an enterprise environment, you must also include a lightweight but concrete responsible-AI and governance approach covering explainability, feedback handling, and accuracy.

Detailed Problem Statement

- E-commerce platforms require a reliable assistant to help users search products, track orders, and place purchases.
- This project focuses on building an AI-powered shopping assistant connected to backend tables such as Products and Orders to fetch accurate information.
- All transactional responses are generated through backend APIs instead of AI guesses. This ensures correctness, transparency, and a trustworthy shopping experience.
- You do not need access to paid APIs or production infrastructure, but you must clearly explain how your design would scale to larger datasets, more users, and stricter enterprise governance requirements.

What You Must Build

- A chat application that allows customers to search available products, track orders, and place purchases, loading/error states, and basic accessibility support.
- Backend services are integrated with database tables such as *Products* and *Orders* to retrieve accurate product details, order status, and purchase confirmations.
- A lightweight backend API that processes requests, runs queries on to backend database, and LLM generates responses based on the data extracted from DB’s.
- Mock tool or function calls (e.g., searchProducts, getOrderStatus) used for all transactional actions, ensuring reliable and auditable responses.
- Implement the Guardrails to protect the internal private data or schemas are not available for the front end.

Example user questions (not limited to):

- “What the t shirt brand Nike available?”
- “What is the status of my order 1234?”
- “When will my order gets delivered?

Required Deliverables

- Implemented source code for the AI-Powered Shopping Assistant application.
- A short note describing prompt design, tool/function usage (if required), and any AI interaction improvements implemented.
- Source code repository with a README with environment covering environment setup, application run commands, and configuration steps for the LLM and dataset.
- Architecture overview (diagram or text)
- Accuracy & limitations note, including risks such as hallucination, incorrect answers, or over-reliance on AI.

Constraints

- The application must be runnable on standard developer hardware using free, publicly available tools; no paid services or special infrastructure are required.
- Use only public datasets like csv to load into local mysql or any other local DB, LLM models, and libraries.
- Any assumptions and limitations must be clearly documented.